— **IC_DisplayedPrecision** defines how many floating point digits are displayed for controls (e.g. sliders). A value of 2, for example, would display a value of 0.523203 as 0.52, even though the precise value is still used internally (and returned if the input is queried). Print will also return .52.

— **IC_Steps** defines how far a slider will move if you click to the left or to the right of its knob. By≈default, this value is derived from the minimum and maximum values that are displayed.

— **INP_Disabled** prevents the user from changing the input's values or moving its preview control widget in the viewer. However, it also prevents calls to Input:SetSource(). If your code has to change such a disabled input you need to temporarily set INP_Disabled to false by calling Input:SetAttrs({INP_Disabled = false})

— **INP_DelayDefault**, if set to true, allows you to define a default value for an input in the OnAddToFlow() function. This is useful if a slider's default value should depend on the composition's frame format or time range as this information isn't available in the Create() function yet.

UI

UI Controls are Sliders, Color selectors and widgets that are used for giving control over variables and on screen positions that are used in the Fuse Plugin and will appear in the Inspector Tool Control area. There are 14 different types of controls.

Add Controls - AddInput

Description

The AddInput function is typically found within the Create event function of a Fuse. It is used to add inputs (controls) to the tool. An input can be one of several control types, or an image type input which appears on the tool tile in the flow.

Usage

self:AddInput(*string* labelname, *string* scriptname, *table* attributes)

labelname (*string*, required)

This string value specifies the label displayed next to the input control. The labelname allows spaces, and so typically is used to present a 'friendly' name for an input control compared to the scripting name described in the second argument.

scriptname (*string*, required)

This string value specifies the name of the input control for purposes of saving the value and for scripting it. The scriptname must not have any spaces, and contain only pure alphanumeric characters.

attributes (*table*, required)

This argument accepts a table of attributes used to define the properties of the input. Minimum and maximum value, whether the tool accepts integer values only, or the options available in a drop down menu are all set within this table. A list of attributes common to most Inputs is displayed below. Attributes specific to a particular input type or preview control will be found in the documentation for that type.

Common Input Attributes

Name	Type : Description
LINKID_DataType	**string :** specifies the type of data produced and used by the control. A control which expects or outputs an image would use type `"Image"` while a slider would use `"Number"`. Valid values include: Image, Number, Point, Text.
LINK_Main	**integer :** specifies the priority or order of LINK style controls. This is used when calculating auto connection of tools in the flow. For example, a tool with two inputs would specify values of 1 and 2 for LINK_Main. The input with a value of 1 would have a higher priority than the control with a value of 2.
INPID_InputControl	**string :** This attribute uses a string to describe the type of control. Valid values include: ButtonControl, CheckboxControl, ColorControl, ComboControl, ComboIDControl, FileControl, FontFileControl, GradientControl, LabelControl, MultiButtonControl, MultiButtonIDControl, OffsetControl, RangeControl, ScrewControl, SliderControl
INPID_PreviewControl	**string :** If present this attribute uses a string to describe the type of on screen control displayed for the control. For example a Point type control would set this attribute to `"TransformControl"` if a set of crosshairs should be displayed in the views when the tool is selected. Valid values include: AngleControl, CrossHairControl, ImgOverlayControl, RectangleControl, PointControl, TransformControl
INP_Default	**numeric :** This attribute is used to set the default value of a control. Typically this will be an integer value.
INP_MinScale	**numeric :** This attribute is used to set the minimum value displayed by the control. Typically this attribute is applied to sliders and range controls. This does not specify the absolute minimum value of the input, only the highest value presented by the control when it is initially constructed. It would still be possible to enter lower values, up to the value specified by INP_MinAllowed.
INP_MaxScale	**numeric :** This attribute is used to set the maximum value displayed by the control. Typically this attribute is applied to sliders and range controls. This does not specify the absolute maximum value of the input, only the highest value presented by the control when it is initially constructed. It would still be possible to enter higher values, up to the value specified by INP_MaxAllowed.
INP_MinAllowed	**numeric :** This attribute specifies the minimum allowable value for the input.
INP_MaxAllowed	**numeric :** This attribute specifies the maximum allowable value for the input.
INP_Required	**boolean :** This attribute specifies whether the input is required. If this is set to false the tool will not fail if the inp is nil. Typically used for non required image inputs - like the foreground of a Merge tool. Defaults to True

Name	Type : Description
IC_ControlGroup	**numeric :** All inputs with the same IC_ControlGroup will be part of an overall group of controls. For example, the Red, Green and Blue sliders of a Color Control would all share the same IC_ControlGroup.
IC_ControlID	**numeric :** A unique identifier for an individual input within a control group defined by IC_ControlGroup. For example, the Red slider in a color control with IC_ControlGroup = 1 would have the IC_ControlID of 0, while the Green slider would have an IC_ControlID of 1, and so on.
IC_Visible	**boolean :** This attribute specifies whether an input is visible. Set it to false to hide the input. Defaults to true.
ICD_Center	**numeric :** This attribute will set the default of a slider to the center, regardless of scale. This creates a non-linear slider, with the default value centered regardless of the MinScale and MaxScale attributes.
ICD_Width	**numeric :** This attribute specifies the width of the input, where a value of 1.0 is the full width of the control page.
PC_Visible	**boolean :** This attribute specifies whether the PreviewControl associated with an input is currently visible. Defaults to true.
PC_GrabPriority	**numeric :** A numeric value that specifies the grab priority of a preview control input. The input with the highest PC_GrabPriority will be selected first. Used when two controls overlap - for example the Angle and Rectangle preview control found in a Merge or Transform tool.
PC_ControlGroup	**numeric :** All inputs with the same PC_ControlGroup will share the same preview control. For example, the Width and Height sliders of a Rectangle mask will have the same PC_ControlGroup.
PC_ControlID	**numeric :** A unique identifier for an individual input within a preview control group defined by PC_ControlGroup. For example, a Width slider for a RectangleControl would have a PC_ControlGroup of 0, while the Height slider would be set to 1.
PC_HideWhileDragging	**boolean :** A true or false value which specifies whether the preview control is visible while it is being dragged.

IC_ControlID

The value of IC_ControlID is sometimes related to a specific image channel. For example, if an X position slider can have its value set by a pick button (see BeginControlNest), it needs to know that it should read the image's "position x" channel. Also, Image:GetChanSize() accepts these to query available image channels. Here's an abbreviated list of valid values, taken from Pixel.h (part of the SDK). Fusion 6.31 or later supports the uppercase channel constants. Earlier versions accept integers only.

ButtonControl

Description

The ButtonControl displays a single clickable button in the tools control window.

This control returns a Number with a value of either 1 (clicked) or 0 (unclicked). To add a Button, set the INPID_InputControl attribute of the AddInput function to the string `"ButtonControl"`.

Checking this Input's value from within the Process dialog will not return a value other than 0. The only time the value is 1 is when the button has been clicked. The button will always immediately return to the unclicked state. For that reason, handling of the button click is generally done from within the NotifyChanged function.

Attributes

Name	Type : Description
BTNC_Align	**string :** This attribute determines the alignment of the button. Valid values are `"Left"`, `"CenteredLeft"`, `"Center"`, `"CenteredRight"`, and `"Right"`.

Example

```
InLabel = self:AddInput("This is a Button", "Label1", {
          LINKID_DataType = "Text",
          INPID_InputControl = "ButtonControl",
          INP_DoNotifyChanged = true,
          INP_External = false,
    })
```

CheckboxControl

Description

The CheckboxControl displays a simple checkbox and label in the tools control window.

This control returns a Number with a value of either 1 (checked) or 0 (unchecked). To add a CheckboxControl, set the INPID_InputControl attribute of the AddInput function to the string "CheckboxControl".

A common usage is to display several checkbox controls on the same line through use of the AddInput functions ICD_Width attribute. It is also generally a good idea to set INP_Integer = true.

Attributes

Name	Type : Description
CBC_TriState	**boolean :** this attribute determines whether the checkbox displays two states or three.

Example

The following shows four checkboxes on the same row of the control window.

```
InR = self:AddInput("Red", "Red", {
    LINKID_DataType = "Number",
    INPID_InputControl = "CheckboxControl",
    INP_Integer = true,
    INP_Default = 1.0,
    ICD_Width = 0.25,
    })

InG = self:AddInput("Green", "Green", {
    LINKID_DataType = "Number",
    INPID_InputControl = "CheckboxControl",
    INP_Integer = true,
    INP_Default = 1.0,
    ICD_Width = 0.25,
    })

InB = self:AddInput("Blue", "Blue", {
    LINKID_DataType = "Number",
    INPID_InputControl = "CheckboxControl",
    INP_Integer = true,
    INP_Default = 1.0,
    ICD_Width = 0.25,
    })

InA = self:AddInput("Alpha", "Alpha", {
    LINKID_DataType = "Number",
    INPID_InputControl = "CheckboxControl",
    INP_Integer = true,
    INP_Default = 1.0,
    ICD_Width = 0.25,
    })
```

ColorControl

Description

The ColorControl adds a dialog used to select a color. It actually consists of several different sliders and button controls combined together.

All of the Inputs associated with a ColorControl return a Text value. To add a ColorControl, set the INPID_InputControl attribute of the AddInput function to the string `"ColorControl"`.

All the Inputs that make up a ColorControl should have the same IC_ControlGroup attribute. The Red slider should have an IC_ControlID of 0, Green is 1, Blue is 2 and Alpha is 3. See the example below.

Note that each of the elements is optional - it is perfectly valid to have a color control that displays only an Alpha slider, for example. Additionally, this control can be used to show more than just RGBA. This can be handy for producing color pickers for other channels in the image.

0=Red, 1=Green, 2=Blue, 3=Alpha, 4=BackgroundR, 5=BackgroundG, 6=BackgroundB. 7=BackgroundA, RealR,RealG,RealB,RealA, 12=Coverage, 13=NormalX, 14=NormalY, 15=NormalZ, 16=Z, 17=U, 18=V, 19=Object, 20=Material.

Attributes

Name	Type : Description
CLRC_ShowWheel	**boolean :** This optional attribute determines whether the color wheel will be displayed.
CLRC_ColorSpace	**string :** This optional attribute is used to specify which color space is used to draw the wheel. Valid options are `"HSV"`, `"HLS"` and `"YUV"`.

Example

```
An example of a full ColorControl.
    InR = self:AddInput("Red", "Red", {
           LINKID_DataType = "Number",
           INPID_InputControl = "ColorControl",
           INP_MinScale = 0.0,
           INP_MaxScale = 1.0,
           INP_Default  = 1.0,
           ICS_Name = "Color",
           IC_ControlGroup = 1,
           IC_ControlID = 0,
           })

    InG = self:AddInput("Green", "Green", {
           LINKID_DataType = "Number",
           INPID_InputControl = "ColorControl",
           INP_MinScale = 0.0,
           INP_MaxScale = 1.0,
           INP_Default  = 1.0,
           IC_ControlGroup = 1,
           IC_ControlID = 1,
           })

    InB = self:AddInput("Blue", "Blue", {
           LINKID_DataType = "Number",
           INPID_InputControl = "ColorControl",
           INP_MinScale = 0.0,
           INP_MaxScale = 1.0,
           INP_Default  = 1.0,
           IC_ControlGroup = 1,
           IC_ControlID = 2,
           })

    InA = self:AddInput("Alpha", "Alpha", {
```

```
            LINKID_DataType = "Number",
            INPID_InputControl = "ColorControl",
            INP_MinScale = 0.0,
            INP_MaxScale = 1.0,
            INP_Default  = 1.0,
            IC_ControlGroup = 1,
            IC_ControlID = 3,
            })
```

ComboControl

Description

The ComboControl presents a **drop down menu**, which returns a Number value. To add a ComboControl, set the INPID_InputControl attribute of the AddInput function to the string `"ComboControl"`.

The return value will be a number representing the position of the selected entry in the ComboControl. The first item in the list will return 0, the second item will return 1, and so on.

Attributes

Name	Type : Description
CC_LabelPosition	**string :** This attribute determines where the label for the ComboControl is drawn. Should be set to either `"Vertical"` or `"Horizontal"`. The default value is `"Horizontal"`
CCS_AddString	**string :** Each time this attribute is entered a new string is added to the ComboControl. As a result, there should be multiple entries of CCS_AddString, each within its own unnamed table index (see example below).

Example

```
The following is an example AddInput function that duplicates the
Operator control on a Merge tool.
    InOperation = self:AddInput("Operator", "Operator", {
            LINKID_DataType = "Number",
            INPID_InputControl = "ComboControl",
            INP_Default = 0.0,
            INP_Integer = true,
            ICD_Width = 0.5,
            { CCS_AddString = "Over", },
            { CCS_AddString = "In", },
            { CCS_AddString = "Held Out", },
            { CCS_AddString = "Atop", },
            { CCS_AddString = "XOr", },
            CC_LabelPosition = "Vertical",
            })
```

ComboIDControl

FileControl

Description

The FileControl adds a dialog used to browse for paths or files on disk. It appears on the tool controls as a text edit box for display and direct entry of filenames, and a button to the right which can be clicked to display the Fusion file browser dialog.

This control returns a Text value. To add a FileControl, set the INPID_InputControl attribute of the AddInput function to the string `"FileControl"`.

Attributes

Name	Type : Description
FC_IsSaver	**boolean :** This attribute determines whether the file dialog may be used to specify files that don't currently exist.
FC_PathBrowse	**boolean :** This attribute is used to specify if the file browse dialog is being used to select a file, or a folder. If it is set to true, the dialog will be used to select folders only. The default value is false.
FC_ClipBrowse	**boolean :** This attribute is used to specify if the file browse dialog is being used to select an image or image sequence. When specified the dialog will be configured with the correct filter to show only known image file types, and the Gather Sequences checkbox will be enabled. The default value is false.
FCS_FilterString	**string :** This attribute is used to specify what file types will be shown by the dialog. The default value is to show All Files, which is equivalent to the filterstring. `"All Files (*.*)|*.*||"`. A more complex filterstring might appear as `"FBX Files (*.fbx)|*.fbx|DAE Files (*.dae)|*.dae|OBJ Files (*.obj)|*.obj|3DS Files (*.3ds)|*.3ds|DXF Files (*.dxf)|*.dxf|"`

Example

```
The following is an example AddInput function that is similar to the
Clip control on a Loader tool.
InFile = self:AddInput("File", "File", {
          LINKID_DataType = "Text",
          INPID_InputControl =  "FileControl",
          FC_ClipBrowse = true,
          })
```

FontFileControl

Description

The FontFileControl adds a dialog used to select Fonts from the list of available fonts on the system. It actually consists of two separate drop down menus. One contains a list of all available fonts, a second displays the list of available styles for the currently selected font.

This control returns a Text value. To add a FontFileControl, set the INPID_InputControl attribute of the AddInput function to the string `"FontFileControl"`.

To create and access both components of the dialog the control should be created twice, with both controls having the same IC_ControlGroup attribute, but with an IC_ControlID of 1 for the Font Name, and of 2 for the Style. See the example below.

Attributes

None

Example

The following is a snippet from a Create function that would create both components of a Font selection dialog.

```
InFont = self:AddInput("Font", "Font", {
	LINKID_DataType = "Text",
	INPID_InputControl = "FontFileControl",
	IC_ControlGroup = 2,
	IC_ControlID = 0,
	INP_Level = 1,
	INP_DoNotifyChanged = true,
})

InFontStyle = self:AddInput("Style", "Style", {
	LINKID_DataType = "Text",
	INPID_InputControl = "FontFileControl",
	IC_ControlGroup = 2,
	IC_ControlID = 1,
	INP_Level = 1,
	INP_DoNotifyChanged = true,
})
```

GradientControl

Description

Gradient Color control has a 1D color ramp. Default Gradient is 2 colors black to white. Use OnAddToFlow to set default colors

Attributes

None

Example

```
InGradient = self:AddInput("Gradient", "Gradient", {
          LINKID_DataType            = "Gradient",
          INPID_InputControl   = "GradientControl",
          INP_DelayDefault           = true,
          })
```

LabelControl

Description

The LabelControl presents a single short text label to the control window for the tool. It does not return any value. To add a LabelControl, set the INPID_InputControl attribute of the AddInput function to the string `"LabelControl"`.

The actual displayed value of the label control is set as the first argument to AddInput

This control should always have the INP_External = false attribute set.

Attributes

None

Example

```
    InLabel = self:AddInput("This is a Label", "Label1", {
          LINKID_DataType = "Text",
          INPID_InputControl = "LabelControl",
          INP_External = false,
          INP_Passive = true,
    })

A LabelControl is also used as a way to toggle the visibility of other
controls. To create such a control nest, use the BeginControlNest
function. It will create a LabelControl with some additional attributes:

      LBLC_DropDownButton = true  -- turns a label control into a
      control nest
      LBLC_NumInputs = 2  -- how many of the following inputs will be
      part of the control nest
      LBLC_NestLevel = 1  -- ?

In addition to control nests, a LabelControl can also host the pick
button. Set LBLC_PickButton = true and specify the picked inputs using
LBLC_NumInputs = <num> (next num inputs), LBLCID_InputGroup = <group
id>, or a series of { LBLCP_Input = <input> } tags.
```

MultiButtonControl

Description

The MultiButtonControl displays one or more buttons used to select from a range of options.

To add a MultibuttonControl to an input, you first specify the MultiButtonControl as the INPID_InputControl attribute in an AddInput function. Each button is then described by a table added to the same AddInput attribute table. See the example below :

This control returns an integer value representing which button is selected. If the first button is selected, the value will be 0, the second button would return as 1, and so forth.

Attributes

Per-button attributes	
MBTNC_AddButton	Set this attribute to a string which will be used as the label for the button.
MBTNCD_ButtonWidth	This should be a fractional value between 0 and 1, determining the width of the button. A value of 1.0 would create a button the full width of the control.
MBTNC_ShowLabel	Enables or disables displaying the button's name text on the button.
MBTNC_ShowIcon	Enables or disables displaying the button's icon on the button.
MBTNCS_ToolTip	Set this to a string to be displayed when hovering the mouse cursor over the button.

Control attributes	
MBTNC_ButtonHeight	This should be an integer value in pixels, determining the height of the buttons. The default value is 26 pixels high.
MBTNC_StretchToFit	Setting this to true will stretch the buttons equally across the full width of the control.
MBTNC_ShowName	Setting this to false will hide the name of the control.
MBTNC_NoIconScaling	Enables or disables scaling of the button's icon to the full size of the button.
MBTNC_Type	Can be one of `"Normal"`, `"Toggle"` or `"TriState"`.
MBTNC_Align	Can be one of `"Center"`, `"Left"` or `"Right"`.

Example

The following fragment shows the creation of a MultiButtonControl with four options.

```
function Create()
    InOperation = self:AddInput("Operation", "Operation", {
            LINKID_DataType = "Number",
            INPID_InputControl = "MultiButtonControl",
```

```
            INP_Default = 0.0,
            { MBTNC_AddButton = "Min", MBTNCD_ButtonWidth = 0.25, },
            { MBTNC_AddButton = "Max", MBTNCD_ButtonWidth = 0.25, },
            { MBTNC_AddButton = "Add", MBTNCD_ButtonWidth = 0.25, },
            { MBTNC_AddButton = "Sub", MBTNCD_ButtonWidth = 0.25, },
            })
```

OffsetControl

Description

The OffsetControl is generally used to represent a 2D coordinate, and presents separate edit boxes for the X and Y coordinates. It is frequently associated with a the Crosshair preview control.

To add an OffsetControl, set the INPID_InputControl attribute of the AddInput function to the string `"OffsetControl"`.

OffsetControl returns the Point datatype.

Attributes

Name	Type : Description
OFCD_DisplayXScale	**numeric :** The value displayed in the offset control for the X axis will be multiplied by the number provided here.
OFCD_DisplayYScale	**numeric :** The value displayed in the offset control for the X axis will be multiplied by the number provided here. See the effect of the Merge and Transform tools reference inputs on the Size input for an example of the effect of these attributes.
INP_DefaultX	**numeric :** The default value to use for the X coordinate.
INP_DefaultY	**numeric :** The default value to use for the Y coordinate.

Example

The following example is a very simplified Fuse which uses a crosshair to transform an image.

```
    FuRegisterClass("SampleOffset", CT_Tool, {
    REGS_Category = "Fuses\\Samples",
    REGS_OpIconString = "SOf",
    REGS_OpDescription = "SampleOffset",
    })

function Create()
    InCenter = self:AddInput("Center", "Center", {
           LINKID_DataType = "Point",
           INPID_InputControl = "OffsetControl",
           INPID_PreviewControl = "CrosshairControl",
           })
```

```
    InImage = self:AddInput("Input", "Input", {
            LINKID_DataType = "Image",
            LINK_Main = 1,
            })

    OutImage = self:AddOutput("Output", "Output", {
            LINKID_DataType = "Image",
            LINK_Main = 1,
            })

end

function Process(req)
    local img     = InImage:GetValue(req)
    local center  = InCenter:GetValue(req)

    out = img:Transform(nil, {
            XF_XOffset = center.X,
            XF_YOffset = center.Y,
            XF_XAxis = 0.5,
            XF_YAxis = 0.5,
            XF_XSize = 1,
            XF_YSize = 1,
            XF_Angle = 0,
            XF_EdgeMode = "Black",
            })

    OutImage:Set(req, out)
end
```

RangeControl

Description

The RangeControl produces a control with a high and low range which can be used to specify a range of value. The control displayed is actually composed of two seperate controls, each of which returns a Number value. To add a RangeControl, create two inputs using the AddInput function. Set the INPID_InputControl attribute to the string `"RangeControl"`, and make sure both controls have the IC_ControlGroup set to the same value.

Set the input which will represent the Low value to have an IC_ControlID of 0, and the input which represents the High value to IC_ControlID of 1. See the example below.

Attributes

Name	Type : Description
RNGCS_LowName	**string :** The label to use on the left (low) side of the control. Overrides the name set by the first argument in the AddInput() function.

Name	Type : Description
RNGCS_MidName	**string :** The label to use in the center of the control. Defaults to blank.
RNGCS_HighName	**string :** The label to use on the right (high) side of the control. Overrides the name set by the first argument in the AddInput() function.
RNGCD_ LowOuterLength	**number :** This attribute can be used to specify the displayed Low limit for the range control. Normally this will be equal to the INP_Default value, or to 0. Lower values can still be entered into the control, this only sets the initial display. Using this attribute will also color the range control yellow.
RNGCD_ HighOuterLength	**number :** This attribute can be used to specify the displayed High limit for the range control. Normally this will be equal to the INP_Default value, or to 1. Higher values can still be entered into the control, this only sets the initial display. Using this attribute will also color the range control yellow.

Example

The following is an example AddInput function that displays a range control.

```
InGLow = self:AddInput("Green Low", "GreenLow", {
       LINKID_DataType = "Number",
       INPID_InputControl = "RangeControl",
       INP_Default = 0.0,
       IC_ControlGroup = 2,
       IC_ControlID = 0,
       })

InGHigh = self:AddInput("Green High", "GreenHigh", {
       LINKID_DataType = "Number",
       INPID_InputControl = "RangeControl",
       INP_Default = 1.0,
       IC_ControlGroup = 2,
       IC_ControlID = 1,
       })
```

Thumbwheel ScrewControl

Description

The ScrewControl presents an infinite slider with no minimum or maximum, with fine control over numbers, typically used to represent values like rotation angles. It returns a Number value. To add a ScrewControl, set the INPID_InputControl attribute of the AddInput function to the string `"ScrewControl"`.

Attributes

None.

Example

```
    InScrewAngle = self:AddInput("Infinite Slider", "ScrewControl", {
            LINKID_DataType = "Number",
            INPID_InputControl = "ScrewControl",
            INP_MinScale = 0.0,
            INP_MaxScale = 100.0,
            INP_Default = 0,
            })
```

SliderControl

Description

The SliderControl presents a simple slider, which returns a Number value. To add a SliderControl, set the INPID_InputControl attribute of the AddInput function to the string `"SliderControl"`.

Attributes

Name	Type : Description
SLCS_LowName	**string :** An optional left justified label that overrides the usual label. Used in conjunction with SLCS_HighName
SLCS_HighName	**string :** An optional right justified label displayed in conjunction with SLCS_LowName. See the Subtractive - Additive slider in the Merge tool for an example of the effect of these attributes.

Example

```
function Create()
    InGain = self:AddInput("Gain", "Gain", {
            LINKID_DataType = "Number",
            INPID_InputControl = "SliderControl",
            INP_Default = 1.0,
            })
```

TextEditControl

Description

The TextEditControl presents a Text input box, which returns a String of Characters.

Attributes

None

Example

```
InTextEntry = self:AddInput("Type Your Text", "Text", {
    LINKID_DataType = "Text",
    INPID_InputControl = "TextEditControl",
    INPS_DefaultText = "hello",  -- use instead of INP_Default!
    TEC_Lines = 3,               -- height of text entry (default is 8)
    TEC_Wrap = true,             -- automatic word-wrapping (default is
    false)
    TEC_ReadOnly = true, -- default is false (you should also set INP_
    External = false)
    TEC_CharLimit = 40,   --maximum number of allowed characters
    (default is 0, no limit)
    TEC_DeferSetInputs = true, -- call NotifyChanged when focus is lost
    (default is
                                  false, call on every keystroke)
    })
```

OnScreen UI Widgets

There are a number of on screen UI widgets for interacting with images, called Preview Controls.

The primary control is the 2D Point Control, other UI widgets link PointControl to other controls.

Point Control

Description

The Point controls are 2D used for on screen manipulation and return 2 values X and Y. These are used in tools like Merge and Transform to position the images.

Attributes

None

Example

```
    InCenter = self:AddInput("Center", "Center", {
           LINKID_DataType      = "Point",
           INPID_InputControl   = "OffsetControl",
           INPID_PreviewControl = "CrosshairControl",
           INP_DefaultX         = 0.5,
           INP_DefaultY         = 0.5,
           })
```

INPID_PreviewControl:

There are 5 different on screen UI Widget used to link other Controls to the Point control, to give added on screen interactions. RectangleControl can be linked to a size slider control.

AngleControl

Description

The AngleControl is an preview control that shows a thin line used to represent the rotation around a specific point. The angle is added to an Input by setting the INPID_PreviewControl attribute to `"AngleControl"` in the AddInput function's attribute table.

Attributes

Name	Type : Description
ACP_Center	**input:** specifies which input is used to set the position of the angle control in the view. Typically this is an OffsetControl
ACP_Radius	**input :** specifies which input is used to set the width of the angle control in the view. Typically this will be a SliderControl.

Example

The following example is a very simplified Fuse which uses a crosshair and angle control to transform an image.

```
FuRegisterClass("SampleAngleControl", CT_Tool, {
    REGS_Name = "Sample Angle Control",
    REGS_Category = "Fuses\\Samples",
    REGS_OpIconString = "SOf",
    REGS_OpDescription = "SampleOffset",
    })

function Create()
    InCenter = self:AddInput("Center", "Center", {
           LINKID_DataType = "Point",
           INPID_InputControl = "OffsetControl",
           INPID_PreviewControl = "CrosshairControl",
           })

    InAngle = self:AddInput("Angle", "Angle", {
           LINKID_DataType = "Number",
           INPID_InputControl = "ScrewControl",
           INPID_PreviewControl = "AngleControl",
           INP_MinScale =   0.0,
           INP_MaxScale = 360.0,
           INP_Default = 0.0,
           ACP_Center = InCenter,
           })

    InImage = self:AddInput("Input", "Input", {
           LINKID_DataType = "Image",
           LINK_Main = 1,
           })
```

```
    OutImage = self:AddOutput("Output", "Output", {
           LINKID_DataType = "Image",
           LINK_Main = 1,
           })

end

function Process(req)
    local img     = InImage:GetValue(req)
    local center  = InCenter:GetValue(req)
    local angle   = InAngle:GetValue(req).Value

    out = img:Transform(nil, {
           XF_XOffset = center.X,
           XF_YOffset = center.Y,
           XF_XAxis = 0.5,
           XF_YAxis = 0.5,
           XF_XSize = 1,
           XF_YSize = 1,
           XF_Angle = angle,
           XF_EdgeMode = "Black",
           })

    OutImage:Set(req, out)
end
```

CrosshairControl

Description

The crosshair control is a preview control that shows a crosshair to represent the position of Point value. The crosshair is added to an Input by setting the INPID_PreviewControl attribute of an input to `"CrosshairControl"` in the AddInput function's attribute table.

There are attributes that apply to all preview controls. PCD_OffsetX and PCD_OffsetY will shift the crosshair in the viewer by the specified distance. This is useful if you want a pivot crosshair to follow the translation crosshair (as is the case with Fusion's transform tool). To implement this behavior, you need to update those attributes in the NotifyChanged() event handler.

Attributes

Name	Type : Description
CHC_Style	**string :** Determines the appearance of the crosshair control. Can be set to `"NormalCross"`, `"DiagonalCross"`, `"Rectangle"`, or `"Circle"`.

Example

The Fuse example above in Point Control demonstrates CrosshairControl.

RectangleControl

Description

The RectangleControl is an on screen preview control which displays a simple rectangle. The RectangleControl is added to an Input by adding the INPID_PreviewControl attribute to the AddInput function's attribute table.

If you use the RCD_LockAspect attribute below, then the width and height are both determined by the same input, and no PC_ControlGroup is needed. Otherwise, a separate input is needed for both the width and height Both inputs should use the same PC_ControlGroup attribute. The input with the attribute PC_ControlID = 0 will determine the width, and the one with PC_ControlID = 1 will determine the height of the Rectangle. Both inputs should have the INPID_PreviewControl set to `"RectangleControl"`.

A slider set to PC_ControlID = 2 can optionally be used to determine the radius of the corners.

Attributes

Name	Type : Description
RCP_Center	**input:** specifies which input is used to set the position of the rectangle control. Typically this is an OffsetControl
RCP_Angle	**input :** specifies which input is used to set the angle of the rectangle control. Typically this will be a ScrewControl.
RCD_LockAspect	**numeric :** If this attribute is set to 1.0 the rectangle control will use the same value for the width and height.
RCD_SetX	**numeric :** A numeric value used to set the position of the rectangle on the X axis.
RCD_SetY	**numeric :** A numeric value used to set the position of the rectangle on the Y axis.

In addition to RCP_Center and RCP_Angle there is also RCP_Axis which needs to point to the input control that defines the center of rotation for the rectangular overlay. In many standard tools, this is an OffsetControl labeled `"Pivot"`.

The transform fuse example that ships with Fusion doesn't completely recreate the rectangle overlay of the regular transform tool. By using RCP_Angle and RCP_Axis, the overlay is transformed correctly. However, the preview controls for angle and pivot don't move along with the rectangle. This needs to be implemented manually:

Turn on INP_DoNotifyChanged for the input control that is linked to RCP_Center.

In the NotifyChanged event handler, set two attributes for both your angle and pivot preview controls (i.e. the inputs linked to RCP_Angle and RCP_Axis): PCD_OffsetX and PCD_OffsetY will shift the preview controls in the viewer and need to be updated with the current position of the center control (minus 0.5 to account for it's default resting position in the middle of the image).

If you want to use separate sliders for X and Y size, keep in mind that PC_Visible (to show/hide the preview widget) also has to be set for both inputs. Otherwise you'll only hide the width or height part of your preview control.

Example

The Create function below shows a rectangle control associated with separate width, height and radius sliders

```
function Create()
    InCenter = self:AddInput("Center", "Center", {
           LINKID_DataType = "Point",
           INPID_InputControl = "OffsetControl",
           INPID_PreviewControl = "CrosshairControl",
           })

    InWidth = self:AddInput("Width", "Width", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INPID_PreviewControl = "RectangleControl",
           PC_ControlGroup = 1.0,
           PC_ControlID = 0,
           INP_MaxScale = 2,
           INP_Default = 1.0,
           })

    InHeight = self:AddInput("Height", "Height", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INPID_PreviewControl = "RectangleControl",
           PC_ControlGroup = 1.0,
           PC_ControlID = 1,
           INP_MaxScale = 2,
           INP_Default = 1.0,
           })

    InRadius = self:AddInput("Corner Radius", "CornerRadius", {
           LINKID_DataType = "Number",
           INPID_InputControl = "SliderControl",
           INPID_PreviewControl = "RectangleControl",
           PC_ControlGroup = 1.0,
           PC_ControlID = 2,
           INP_MaxScale = 0.2,
           INP_Default = 0.0,
           })

    InAngle = self:AddInput("Angle", "Angle", {
           LINKID_DataType = "Number",
           INPID_InputControl = "ScrewControl",
           INPID_PreviewControl = "AngleControl",
           INP_MinScale =   0.0,
           INP_MaxScale = 360.0,
```